

Day 3, Session 3: Shift Registers

Concept Block (12 min)

What is a Shift Register?

A shift register is a chain of flip-flops where data "moves" from one flip-flop to the next on each clock cycle.

Visual analogy:

- Think of a **conveyor belt** in a factory
- Each flip-flop is a "station" on the belt
- On each clock cycle, data moves one station over
- New data enters at one end, old data exits at the other

Simple 4-bit shift register:

Clock 0: [0] [0] [0] [0]

↑

Input: 1

Clock 1: [1] [0] [0] [0] ← Data shifted right

↑

Input: 1

Clock 2: [1] [1] [0] [0] ← Data shifted right again

↑

Input: 0

Clock 3: [0] [1] [1] [0] ← Data continues shifting

↑

Input: 1

Clock 4: [1] [0] [1] [1]

↓

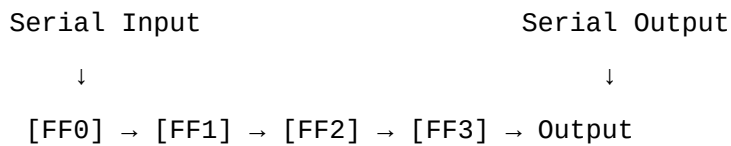
Output: 1 (pushed out)

Key concept: Data "flows" through the register like water through a pipe.

The Four Types of Shift Registers

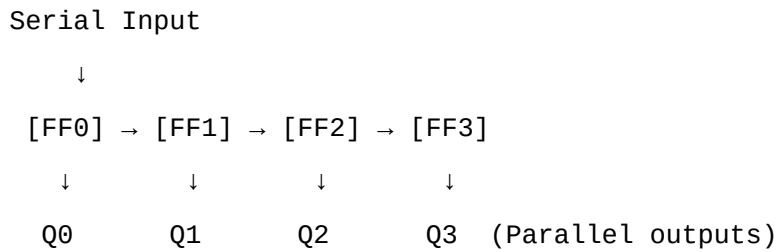
Shift registers are classified by their **input/output style**:

1. SISO (Serial In, Serial Out)



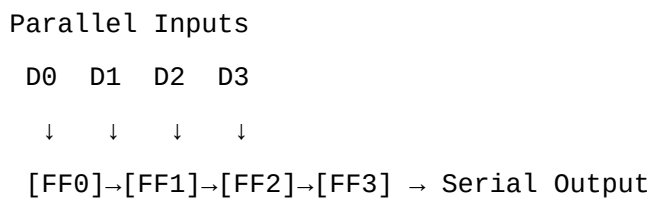
- **Input:** One bit at a time (serial)
- **Output:** One bit at a time (serial)
- **Use:** Delay lines, timing circuits

2. SIPO (Serial In, Parallel Out)



- **Input:** One bit at a time (serial)
- **Output:** All bits available simultaneously (parallel)
- **Use:** Serial-to-parallel converter (UART receiver, SPI)

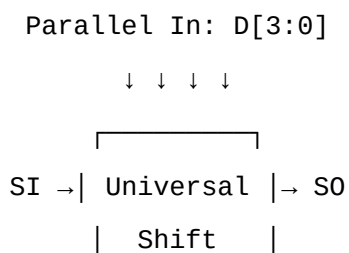
3. PISO (Parallel In, Serial Out)



- **Input:** All bits loaded at once (parallel)
- **Output:** One bit at a time (serial)
- **Use:** Parallel-to-serial converter (UART transmitter, SPI)

4. Universal Shift Register

- Control signals:
- Load (parallel)
 - Shift Left/Right
 - Enable



| Register |

└─────────┘

↓ ↓ ↓ ↓

Parallel Out: Q[3:0]

- **Combines all modes**
- **Control signals** select operation
- **Use:** Flexible data manipulation, barrel shifters

Why Shift Registers Exist

Real-world applications:

1. Serial Communication (UART, SPI, I²C)

- CPUs work with **parallel data** (8-bit, 16-bit buses)
- Serial protocols send **one bit at a time** (fewer wires)
- **PISO:** Convert parallel data to serial for transmission
- **SIPO:** Convert received serial data back to parallel

2. LED Matrix / Seven-Segment Displays

- Controlling 8 LEDs needs 8 pins
- With shift register: Use only 3 pins (data, clock, latch)
- Shift data in serially, then display in parallel

3. Delay Lines / Audio Effects

- Shift register creates **exact delays** (N clock cycles)
- Used in echo, reverb, chorus effects

4. Data Alignment / Bit Manipulation

- Extract specific bits from data stream
- Align data to byte boundaries

5. CRC / Checksum Calculation

- Linear Feedback Shift Registers (LFSR)
- Used in network protocols, error detection

6. Pseudo-Random Number Generation

- LFSR with feedback taps creates pseudo-random sequences
- Used in cryptography, simulations

Shift Direction: Left vs Right

Visual representation:

RIGHT SHIFT (most common):

MSB ← ← ← ← LSB

[D3] [D2] [D1] [D0]

← ← ← ← (Serial In)

LEFT SHIFT:

MSB → → → → LSB

[D3] [D2] [D1] [D0]

→ → → → (Serial In)

Convention:

- **Right shift:** Data moves from higher index to lower (D3 → D2 → D1 → D0)
- **Left shift:** Data moves from lower index to higher (D0 → D1 → D2 → D3)

In arithmetic:

- Left shift = multiply by 2 (binary: 0101 << 1 = 1010)
- Right shift = divide by 2 (binary: 1010 >> 1 = 0101)

Code & Practical Block (15 min)

Design 1: SISO (Serial In, Serial Out)

// siso.v - Serial In, Serial Out Shift Register (4-bit)

```
module siso (  
    input  wire clk,          // Clock  
    input  wire rst,          // Async reset  
    input  wire si,           // Serial input (one bit)  
    output wire so             // Serial output (one bit)  
);  
  
    reg [3:0] shift_reg;      // 4-bit internal shift register  
  
    // Shift operation  
    always @(posedge clk or posedge rst) begin  
        if (rst)  
            shift_reg <= 4'b0000;          // Clear register  
        else  
            shift_reg <= {shift_reg[2:0], si}; // Shift left, insert si at LSB  
    end  
  
    // Serial output is MSB  
    assign so = shift_reg[3];
```

endmodule

Line-by-Line: SISO Implementation

Line 10: `reg [3:0] shift_reg;`

- Internal 4-bit storage
- Holds the data being shifted

Line 16: `shift_reg <= {shift_reg[2:0], si};`

- **THIS IS THE KEY LINE** - let me break it down:

Concatenation operator {}:

`{shift_reg[2:0], si}`

- `shift_reg[2:0]`: Take bits 2, 1, 0 (bottom 3 bits)
- `si`: Serial input (1 bit)
- `{}`: Concatenate them together

What happens:

Before shift: `shift_reg = [D3][D2][D1][D0]`

↑
Serial In = X

After shift: `shift_reg = [D2][D1][D0][X]`

↑
New bits from right

Step-by-step example:

Initial: `shift_reg = 4'b0000, si = 1`

Cycle 1:

```
shift_reg[2:0] = 3'b000
{3'b000, 1'b1} = 4'b0001
shift_reg <= 4'b0001
```

Cycle 2: `si = 1`

```
shift_reg[2:0] = 3'b001
{3'b001, 1'b1} = 4'b0011
shift_reg <= 4'b0011
```

Cycle 3: `si = 0`

```
shift_reg[2:0] = 3'b011
```

```
{3'b011, 1'b0} = 4'b0110
shift_reg <= 4'b0110
```

Cycle 4: si = 1

```
shift_reg[2:0] = 3'b110
{3'b110, 1'b1} = 4'b1101
shift_reg <= 4'b1101
```

Data shifted from RIGHT to LEFT (serial input enters at bit 0, exits at bit 3).

Line 20: assign so = shift_reg[3];

- Serial output is the **MSB** (bit 3)
 - This is the bit that gets "pushed out" after 4 shifts
-

Design 2: SIPO (Serial In, Parallel Out)

```
// sipo.v - Serial In, Parallel Out Shift Register (8-bit)
module sipo (
    input wire      clk,      // Clock
    input wire      rst,      // Async reset
    input wire      si,       // Serial input
    output reg [7:0] po       // Parallel output (all 8 bits)
);

    always @(posedge clk or posedge rst) begin
        if (rst)
            po <= 8'b0;
        else
            po <= {po[6:0], si}; // Shift left, insert si at LSB
    end

endmodule
```

Key difference from SISO:

- Output is po[7:0] - **all 8 bits** available simultaneously
 - No separate internal register - po itself is the shift register
 - Used when you need to **receive 8 bits serially, then use them as a byte**
-

Design 3: PISO (Parallel In, Serial Out)

```
// piso.v - Parallel In, Serial Out Shift Register (8-bit)
module piso (
    input wire      clk,      // Clock
    input wire      rst,      // Async reset
    input wire      load,     // Load signal (parallel load when HIGH)
    input wire [7:0] pi,      // Parallel input
    output wire      so       // Serial output
);

    reg [7:0] shift_reg;

    always @(posedge clk or posedge rst) begin
        if (rst)
            shift_reg <= 8'b0;
        else if (load)
            shift_reg <= pi;           // Parallel load
        else
            shift_reg <= {shift_reg[6:0], 1'b0}; // Shift left, fill with 0
    end

    assign so = shift_reg[7]; // Serial output from MSB

endmodule
```

Line-by-Line: PISO Implementation

Lines 14-18: Priority logic

```
if (rst)
    shift_reg <= 8'b0;           // Priority 1: Reset
else if (load)
    shift_reg <= pi;             // Priority 2: Parallel load
else
    shift_reg <= {shift_reg[6:0], 1'b0}; // Priority 3: Shift
```

Priority: Reset > Load > Shift

Operation modes:

1. **Load mode (load=1):** All 8 bits of `pi` loaded in parallel

2. **Shift mode (load=0):** Data shifts left, LSB filled with 0

Line 17: `shift_reg <= {shift_reg[6:0], 1'b0};`

- Take bottom 7 bits, append a 0 at LSB
- MSB gets "pushed out" and appears on SO

Typical usage pattern:

Cycle 0: `load=1, pi=8'b10110101` → `shift_reg = 10110101`

Cycle 1: `load=0` → `shift_reg = 01101010, so=1`

Cycle 2: `load=0` → `shift_reg = 11010100, so=0`

Cycle 3: `load=0` → `shift_reg = 10101000, so=1`

...

Testbench: SISO Example

`// siso_tb.v - Testbench for SISO Shift Register`

```
module siso_tb;
```

```
    reg clk;
```

```
    reg rst;
```

```
    reg si;
```

```
    wire so;
```

```
    // Instantiate SISO
```

```
    siso uut (
```

```
        .clk(clk),
```

```
        .rst(rst),
```

```
        .si(si),
```

```
        .so(so)
```

```
    );
```

```
    // Clock generator
```

```
    initial begin
```

```
        clk = 0;
```

```
        forever #5 clk = ~clk;
```

```
    end
```

```
    // Test stimulus
```

```
    initial begin
```

```
        $dumpfile("siso.vcd");
```



```

$dumpvars(0, siso_tb);

$display("Time | clk | rst | si | so | shift_reg");
$display("-----");
$monitor("%4t | %b | %b | %b | %b | %b",
         $time, clk, rst, si, so, uut.shift_reg);

// Reset
rst = 1; si = 0;
#10;
rst = 0;
#10;

// Shift in pattern: 1011 (one bit per clock)
$display("\n--- Shifting in 1011 ---");
si = 1; #10;    // Bit 0: 1
si = 0; #10;    // Bit 1: 0
si = 1; #10;    // Bit 2: 1
si = 1; #10;    // Bit 3: 1

// Shift in zeros (to push data out)
$display("\n--- Shifting out ---");
si = 0; #10;
si = 0; #10;
si = 0; #10;
si = 0; #10;

$finish;
end

endmodule

```

Expected SISO Behavior

Time	clk	rst	si	so	shift_reg

0	0	1	0	0	0000 ← Reset

10		0		0		0		0		0000	← Reset released
20		0		0		1		0		0000	← si=1 applied
25		1		0		1		0		0001	← Shift: 1 enters at LSB
30		0		0		0		0		0001	
35		1		0		0		0		0010	← Shift: 0 enters
40		0		0		1		0		0010	
45		1		0		1		0		0101	← Shift: 1 enters
50		0		0		1		0		0101	
55		1		0		1		1		1011	← Shift: 1 enters, MSB becomes 1

↑

First '1' reaches output!

Pattern in register: 0000 → 0001 → 0010 → 0101 → 1011

After 4 shifts, the pattern 1011 is fully loaded.

Compile and Test

```
iverilog -o sim/siso siso.v siso_tb.v
```

```
vvp sim/siso
```

```
gtkwave siso.vcd
```

In GTKWave:

- Watch `shift_reg` - you'll see data "move" from right to left
- Watch `so` - it outputs the MSB (bit that gets pushed out)

Mini Task (8 min)

Task: Build a **SIPO (Serial In, Parallel Out)** testbench and verify correct operation.

Requirements:

1. Use the `sipo` module provided above
2. Shift in the 8-bit pattern: 10110011 (one bit per clock)
3. After 8 shifts, verify `po` contains 10110011
4. Display each shift showing how `po` builds up

Testbench template:

```
module sipo_tb;
```

```
    reg          clk, rst, si;
```

```
    wire [7:0] po;
```

```
    sipo uut (
```

```

        .clk(clk),
        .rst(rst),
        .si(si),
        .po(po)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        $dumpfile("sipo.vcd");
        $dumpvars(0, sipo_tb);

        $display("Time | si | po");
        $display("-----");
        $monitor("%4t | %b | %b", $time, si, po);

        // TODO: Reset, then shift in 10110011 bit by bit
        // Order: LSB first (1, 1, 0, 0, 1, 1, 0, 1)

        $finish;
    end
endmodule

```

Hint: Shift in **LSB first**: Pattern 10110011 means send bits in order: 1, 1, 0, 0, 1, 1, 0, 1

Submit your SIPO testbench and show me the output.

Once verified, we'll build **PISO** and then the **Universal Shift Register** (combines all modes!).

Universal Shift Register - The Ultimate Shift Register!

Excellent choice! This is the **most powerful and flexible** shift register design. Let's build it properly.

Concept: What Makes It "Universal"?

A universal shift register can do EVERYTHING:

1. **Hold** - Maintain current value
2. **Shift Left** - Data moves toward MSB
3. **Shift Right** - Data moves toward LSB
4. **Parallel Load** - Load all bits at once
5. **Rotate** - Shift with wraparound (optional)

All controlled by mode selection signals!

Design Specification

Control Signals (Mode Selection)

We'll use a **2-bit mode** signal:

mode[1:0]:

- 00 = Hold (no operation)
- 01 = Shift Right (MSB → LSB direction)
- 10 = Shift Left (LSB → MSB direction)
- 11 = Parallel Load

Port Definitions

Inputs:

- clk, rst: Clock and async reset
- mode[1:0]: Operation mode selector
- si: Serial input (used for shifting)
- pi[7:0]: Parallel input (for parallel load)

Outputs:

- po[7:0]: Parallel output (always visible)
 - so_left: Serial output (MSB - for left shift)
 - so_right: Serial output (LSB - for right shift)
-

Implementation: Universal Shift Register

// universal_shift_reg.v - Universal Shift Register (8-bit)

```
module universal_shift_reg (  
    input  wire      clk,          // Clock  
    input  wire      rst,          // Async reset  
    input  wire [1:0] mode,        // Mode: 00=hold, 01=right, 10=left, 11=load
```

```

input wire      si,          // Serial input
input wire [7:0] pi,         // Parallel input
output reg  [7:0] po,        // Parallel output
output wire   so_left,       // Serial output (MSB)
output wire   so_right      // Serial output (LSB)
);

// Mode definitions (for readability)
localparam HOLD      = 2'b00;
localparam SHIFT_RIGHT = 2'b01;
localparam SHIFT_LEFT  = 2'b10;
localparam LOAD       = 2'b11;

// Shift register operation
always @(posedge clk or posedge rst) begin
    if (rst)
        po <= 8'b0;          // Reset: clear register
    else begin
        case (mode)
            HOLD:      po <= po;          // Hold current value
            SHIFT_RIGHT: po <= {si, po[7:1]}; // Shift right, insert
si at MSB
            SHIFT_LEFT:  po <= {po[6:0], si}; // Shift left, insert si
at LSB
            LOAD:      po <= pi;          // Parallel load
            default:    po <= po;          // Default: hold
        endcase
    end
end

// Serial outputs (combinational)
assign so_left  = po[7]; // MSB for left shift output
assign so_right = po[0]; // LSB for right shift output

endmodule

```

Line-by-Line Explanation

Lines 15-18: Mode Definitions (localparam)

```
localparam HOLD          = 2'b00;
localparam SHIFT_RIGHT = 2'b01;
localparam SHIFT_LEFT  = 2'b10;
localparam LOAD         = 2'b11;
```

What is localparam?

- Like parameter, but **local to the module** (cannot be overridden)
- Used for **constants** that improve code readability
- Makes code self-documenting

Why use it?

- Writing `mode == SHIFT_LEFT` is clearer than `mode == 2'b10`
- If you change encoding, you only update one place

Lines 25-32: Case Statement (Mode Selection)

```
case (mode)
    HOLD:          po <= po;                // Hold
    SHIFT_RIGHT:   po <= {si, po[7:1]};      // Right shift
    SHIFT_LEFT:    po <= {po[6:0], si};      // Left shift
    LOAD:          po <= pi;                // Load
    default:       po <= po;                // Default
endcase
```

Let's break down each operation:

1. HOLD (mode = 00)

```
po <= po;
```

- Assign `po` to itself
- Register maintains current value
- No change on clock edge

2. SHIFT RIGHT (mode = 01)

```
po <= {si, po[7:1]};
```

What this does:

- `po[7:1]`: Take bits 7 down to 1 (top 7 bits)
- `si`: Serial input (1 bit)
- `{si, po[7:1]}`: Concatenate `si` as MSB, followed by top 7 bits

Visual representation:

Before: `po = [D7][D6][D5][D4][D3][D2][D1][D0]`

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓

After: `po = [si][D7][D6][D5][D4][D3][D2][D1]`

↑ ↓

New bit enters

D0 pushed out (to `so_right`)

Data flows: `si` → **MSB** → ... → **LSB** → `so_right`

3. SHIFT LEFT (mode = 10)

```
po <= {po[6:0], si};
```

What this does:

- `po[6:0]`: Take bits 6 down to 0 (bottom 7 bits)
- `si`: Serial input
- `{po[6:0], si}`: Concatenate bottom 7 bits as MSB, `si` as LSB

Visual representation:

Before: `po = [D7][D6][D5][D4][D3][D2][D1][D0]`

↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓

After: `po = [D6][D5][D4][D3][D2][D1][D0][si]`

↓ ↑

D7 pushed out (to `so_left`)

New bit enters

Data flows: `so_left` ← **MSB** ← ... ← **LSB** ← `si`

4. PARALLEL LOAD (mode = 11)

```
po <= pi;
```

- Load all 8 bits from `pi` in one clock cycle
 - Same as PISO behavior
-

Lines 36-37: Serial Outputs

```
assign so_left = po[7];    // MSB
```

```
assign so_right = po[0];   // LSB
```

Why two serial outputs?

- `so_left`: When shifting left, data exits from MSB (bit 7)
- `so_right`: When shifting right, data exits from LSB (bit 0)

Use case:

- Left shift → Use `so_left` to capture shifted-out bits
- Right shift → Use `so_right` to capture shifted-out bits

Testbench: Universal Shift Register

```
// tb_universal_shift_reg.v
module tb_universal_shift_reg;

    reg        clk, rst;
    reg  [1:0] mode;
    reg        si;
    reg  [7:0] pi;
    wire [7:0] po;
    wire        so_left, so_right;

    // Mode definitions (match module)
    localparam HOLD          = 2'b00;
    localparam SHIFT_RIGHT = 2'b01;
    localparam SHIFT_LEFT  = 2'b10;
    localparam LOAD         = 2'b11;

    universal_shift_reg uut (
        .clk(clk),
        .rst(rst),
        .mode(mode),
        .si(si),
        .pi(pi),
        .po(po),
        .so_left(so_left),
        .so_right(so_right)
    );

    // Clock generator
    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    // Test stimulus
```



```

initial begin
    $dumpfile("universal_shift_reg.vcd");
    $dumpvars(0, tb_universal_shift_reg);

    $display("Time | mode | si | pi          | po          | so_L | so_R");
    $display("-----");
    $monitor("%4t | %b   | %b   | %b | %b | %b   | %b",
              $time, mode, si, pi, po, so_left, so_right);

    // Initialize
    rst = 1; mode = HOLD; si = 0; pi = 8'b0;
    #10;
    rst = 0;
    #10;

    // Test 1: Parallel Load
    $display("\n--- Test 1: Parallel Load ---");
    pi = 8'b101110011;
    mode = LOAD;
    #10;

    // Test 2: Hold (should maintain value)
    $display("\n--- Test 2: Hold ---");
    mode = HOLD;
    pi = 8'b11111111; // Change pi, but should be ignored
    #20;

    // Test 3: Shift Right (si=1, then si=0)
    $display("\n--- Test 3: Shift Right ---");
    mode = SHIFT_RIGHT;
    si = 1; #10;
    si = 0; #10;
    si = 1; #10;

    // Test 4: Reload and Shift Left
    $display("\n--- Test 4: Parallel Load + Shift Left ---");

```

```

mode = LOAD;
pi = 8'b11001100;
#10;
mode = SHIFT_LEFT;
si = 1; #10;
si = 0; #10;
si = 1; #10;

// Test 5: Shift Right multiple times
$display("\n--- Test 5: Shift Right (push all bits out) ---");
mode = LOAD;
pi = 8'b10101010;
#10;
mode = SHIFT_RIGHT;
si = 0;
repeat (8) #10; // Shift 8 times

$finish;
end

endmodule

```

Expected Behavior

Test 1: Parallel Load

mode=11 (LOAD), pi=10110011

→ po becomes 10110011 immediately

Test 2: Hold

mode=00 (HOLD), pi changes to 11111111

→ po stays 10110011 (ignores pi)

Test 3: Shift Right

mode=01 (SHIFT_RIGHT), po=10110011

si=1 → po=11011001 (1 inserted at MSB)

si=0 → po=01101100 (0 inserted at MSB)

si=1 → po=10110110 (1 inserted at MSB)

Test 4: Shift Left

mode=11 (LOAD), pi=11001100

→ po=11001100

mode=10 (SHIFT_LEFT)

si=1 → po=10011001 (1 inserted at LSB)

si=0 → po=00110010 (0 inserted at LSB)

si=1 → po=01100101 (1 inserted at LSB)

Compile and Test

```
iverilog -o sim/universal_shift_reg universal_shift_reg.v
```

```
tb_universal_shift_reg.v
```

```
vvp sim/universal_shift_reg
```

```
gtkwave universal_shift_reg.vcd
```

Mini Task: Verify All Modes

Run the simulation and verify:

1. ☒ **HOLD mode:** po doesn't change even when pi changes
2. ☒ **SHIFT_RIGHT:** Data moves from MSB → LSB, si enters at MSB
3. ☒ **SHIFT_LEFT:** Data moves from LSB → MSB, si enters at LSB
4. ☒ **LOAD:** Parallel load works instantly

Build and test the universal shift register. Show me the output!

Once verified, **Day 3 Session 3 is COMPLETE!** Then we move to **Day 3 Session 4: Advanced Counters** (up-down, modulo-N, with load/enable). 🚀

This universal shift register is **production-quality** code. Understanding this design proves you've mastered shift register concepts! 🎯